

```

Vuelo ES seq < IR >
millasGanadas ES seq < struct < millas: Millas, nombrePromo: string, factorPromo: IR > >
millasCanjeadas ES seq < IR >
millasRecibidas ES seq < struct < transferidos: Usuario, millas: Millas > >
millasTransferidas ES seq < struct < receptor: Usuario, millas: Millas > >

enum Categoria ES { Bronce, Plata, Oro, Platino }

```

```

Transacciones ES struct < distanciaVolada, millasGanadas, millasCanjeadas, millasRecibidas, millasTransferidas >.

```

```

Usuarios ES dicc < usuario: Usuario, transacciones: Transacciones >

```

```

Características ES struct < factor: IR, tope: IR >

```

```

PromoConTope ES dicc < nombre: string, características: Características >

```

```

Promos ES struct < nombre: string, factor: IR >

```

```

Usuario ES  $\mathbb{Z}$ 

```

```

millas ES IR

```

```

PromosConTope ES seq < PromoConTope >

```

```

    dicc < nombre: promo, factor: IR >

```

```

TAD AirMiles {

```

```

    obs usuarios: Usuarios

```

```

    obs promos: Promos

```

```

    obs promosConTope: PromosConTope

```

```

proc clearSistema(): AirMiles

```

```

    requiere: { True }

```

```

    asegura: { res.usuarios = {}  $\wedge$  res.promos = seq << nombre: 'NoPromo', factor: 1 >>  $\wedge$  res.promosConTope = <> }

```

```

proc registrarUsuario (inout aerolínea: AirMiles, in usuario: Usuario) {

```

```

    requiere: { usuario  $\notin$  aerolínea.usuarios  $\wedge$  aerolínea = aerolínea0

```

```

    asegura: { |aerolínea.usuarios| = |aerolínea0.usuarios| + 1  $\wedge$ 

```

```

    aerolínea.usuarios = SetKey(aerolínea.usuarios0, usuario, < vuelos: <>, millasGanadas: <>, millasCanjeadas: <>,
    millasRecibidas: <>, millasTransferidas: <> > )  $\wedge$ 

```

```

    aerolínea.promos = aerolínea0.promos  $\wedge$  aerolínea.promosConTope = aerolínea0.promosConTope

```

```

proc registrarUsuarioConReferidor (inout aerolinea: AirMiles, in usuario: Usuario, in referidor: Usuario) {
  requiere { aerolinea = aerolinea0  $\wedge$  usuario  $\notin$  aerolinea.usuarios  $\wedge$  referidor  $\in$  aerolinea.usuarios }

```

```

  asegura { |aerolinea.usuarios| = |aerolinea.usuarios| + 1  $\wedge$ 
  aerolinea.usuarios = setkey(aerolinea0.usuario, usuario,
    < vuelos: < >,

```

```

    millasGanadas: < 500, nombrePromo: aerolinea0.usuarios[referidor].millasGanadas.nombrePromo, factorPromo: aerolinea0.usuarios[referidor].millasGanadas.factorPromo,
    millasCanjeadas: < >, millasRecibidas: < >, millasTransferidas: < > }

```

millas Ganadas

```

 $\wedge$  aerolinea.usuarios = setkey(agregarUsuario(aerolineas0.usuarios, usuario), referidor, < vuelos: aerolinea0.usuarios[referidor].vuelos

```

```

 $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoConTope = aerolinea0.promoConTope

```

```

aux agregarUsuarioReferido (usuarios: Usuarios, usuario: Usuario) : Usuarios

```

```

  setkey (usuarios, usuario, < vuelos: < >, millasGanadas: < millas: 500, nombrePromo: 'NoPromo', factorPromo: 1 >, millasCanjeadas: < >,
    millasRecibidas: < >, millasTransferidas >

```

```

  ifThenElse ( promo  $\in$  aerolinea.promos, aerolinea.usuarios = acumularMillasAux(  $\#$  ), aerolinea.usuarios = acumularMillasConTope(

```

```

     $\#$  ( aerolinea0.usuarios, usuario )

```

```

  aux acumularMillasConTope (
    ifThenElse ( topeEsSuperado, actualizarSistema, acumularMAux )

```

```

proc acumularMillas ( inout aerolínea : AirMiles , in usuario : Usuario , in distanciaVolada : Vuelo , in promo : string )
  requiere : { aerolínea = aerolínea0 ∧ usuario ∈ aerolínea.usuarios ∧ ( promo ∈ aerolínea.promos ∨ promo ∈ aerolínea.promosConTope )
               ∧ distanciaVolada > 0 }

```

```

asegura { aerolínea.usuarios = agregarNuevaTransacción ( aerolínea0.usuarios , usuario ) ∧
          aerolínea.promosConTope = actualizarPromoConTope ( promo , aerolínea.usuarios [ usuario ] , aerolínea.promosConTope )
          ∧ aerolínea.promos = aerolínea0.promos }

```

```

aux agregarMillasGanadas ( distanciaVolada : Vuelo , transacciones : Transacciones , promo : string , factor : IR ) : MillasGanadas

```

```

  < Millas : calcularMillasAcumuladas ( distanciaVolada , transacciones , promo ) , nombrePromo : promo , factorPromo : factor >

```

```

aux calcularMillasAcumuladas ( distanciaVolada : Vuelo , transacciones : Transacciones , factorPromo : IR ) : IR
  distanciaVolada * factorCategoría ( transacciones ) * factorPromo

```

```

pred esBronce ( millasTotales : IR )
  millasTotales < 10.000

```

```

pred esPlata ( millasTotales : IR )
  10.000 ≤ millasTotales < 50.000

```

```

pred esOro ( millasTotales : IR )
  50.000 ≤ millasTotales < 1.000.000

```

```

aux factorCategoría ( transacciones : Transacciones )

```

```

IfThenElse ( esBronce ( millasTotales ( transacciones ) , 1.0 , IfThenElse ( esPlata ( millasTotales ( transacciones ) , 1.5 ,
                                                                                               IfThenElse ( esOro ( millasTotales ( transacciones ) , 2.0 , 0 ) ) ) ) ) )

```

aux agregarNuevaTransacción (usuarios: Usuarios, usuario: Usuario) : Usuarios
 Setkey (usuarios , usuario , < concat (usuarios[usuario].vuelos , < distanciaVolada> ,
 concat (usuarios[usuario].millas Ganadas , < agregarMillas Ganadas (usuarios[usuario] , promo , distVolada)> ,
 millas Ganadas : usuarios[usuario].millas Canjeadas ,
 usuarios[usuario].millas Recibidas ,
 usuarios[usuario].millas Transferidas > .

aux millasAcumuladas (transacciones: Transacciones , promo: string)
 |millasGanadas|
 $\sum_{i=0} \text{ifThenElse (millasGanadas[i].nombrePromo = promo , millasGanadas[i].millas , 0)}$

preol esTopeSuperado (transacciones: Transacciones , tope: R)
 tope > millasAcumuladasAux (transacciones)

aux eliminarPromoContoPe (promo

aux actualizarPromos (promo: string , usuarios: Usuarios , promosContoPe : PromosContoPe) :
 ifThenElse (promo ∈ promosContoPe , evaluarTope () , promosContoPe)

ifThenElse (acum > tope , eliminarPromosContoPe () , promosContoPe

```

proc canjearMillas (inout aerolinea : AirMiles, usuario : Usuario, millasACanjar : Millas)
  requiere { aerolinea = aerolinea0  $\wedge$  usuario  $\in$  aerolinea.usuarios  $\wedge$  millasACanjar  $\leq$  millasDisponibles (aerolinea0.usuarios[usuario]) }
  asegura { aerolinea.usuarios = SetKey (aerolinea0.usuarios, usuario, < Vuelos: aerolinea0.usuarios[usuario].Vuelos,
    MillasGanadas: // // . MillasGanadas,
    MillasCanjeadas: Concat (aerolinea0.usuarios[usuario].MillasCanjeadas, <millasACanjar>),
    MillasRecibidas: // // . MillasRecibidas,
    MillasTransferidas: // // . MillasTransferidas > )
     $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoContope = aerolinea0.promoContope }

```

```

proc transferirMillas (inout aerolinea : AirMiles, in usuarioTransferidor : Usuario, in usuarioReceptor : Usuario, in millasATransferir : Millas)
  requiere { aerolinea = aerolinea0  $\wedge$  usuarioTransferidor  $\in$  aerolinea.usuarios  $\wedge$  usuarioReceptor  $\in$  claves (aerolinea.usuarios)  $\wedge$ 
    millasATransferir  $\leq$  millasDisponibles (aerolinea0.usuarios[usuarioTransferidor]) }
  asegura { aerolinea.usuarios = SetKey (agregarTransferenciaRecibida (aerolinea0.usuarios, usuarioTransferidor, usuarioReceptor, millasATransferir),
    usuarioTransferidor, < Vuelos: usuarios[usuarioReceptor].Vuelos,
    millasGanadas: // // . MillasGanadas
    millasCanjeadas: // // . MillasCanjeadas
    millasRecibidas: // // . MillasRecibidas
    millasTransferidas: Concat (usuarios[usuarioTransferidor].MillasTransferidas, <receptor: usuarioReceptor,
    millasTransferidas: millasATransferir > )
     $\wedge$  aerolinea.promo = aerolinea0.promo  $\wedge$  aerolinea.promoContope = aerolinea0.promoContope }

```

```

aux agregarTransferenciaRecibida (usuarios: Usuarios, usuarioTransferidor : Usuario, usuarioReceptor : Usuario, millasATransferir : Millas) =
  setKey (usuarios, usuarioReceptor, < Vuelos: usuarios[usuarioReceptor].Vuelos,
    millasGanadas: // // . MillasGanadas
    millasCanjeadas: // // . MillasCanjeadas
    millasRecibidas: Concat (usuarios[usuarioReceptor].MillasRecibidas, <transferidor: usuarioTransferidor,
    millasTransferidas: // // . MillasTransferidas millasRecibidas: millasATransferir > )

```

proc consultarSaldo (in aerolínea: AirMiles, in usuario: Usuario) : Millas
 requiere : { usuario $\in$ aerolínea.usuarios
 asegura : { res = millasDisponibles (aerolínea.usuarios[usuario])

proc consultarCategoria (in aerolínea: AirMiles, usuario: Usuario) : Categoría
 requiere : { usuario $\in$ aerolínea.usuarios
 asegura : { res = IfThenElse (esBronce (millasTotales (transacciones), Bronce ,
 IfThenElse (esPlata (millasTotales (transacciones), Plata ,
 IfThenElse (esOro (millasTotales (transacciones) , Oro , Platino))) }

proc rankingMillas (in aerolínea: AirMiles, in cantidad: $\mathbb{Z}$) : Seq < Usuario >
 requiere : { cantidad > 0 }
 asegura : { |res| = cantidad $\Leftrightarrow$ cantidad $\leq$ |aerolínea.usuarios| $\wedge$
 |res| = |aerolínea.usuarios| $\Leftrightarrow$ cantidad > |aerolínea.usuarios|
 ($\forall i: \mathbb{Z}) (0 \leq i < |res| - 1 \rightarrow \text{res}[i] \in \text{aerolínea.usuarios} \wedge \text{millasDisponibles}(\text{aerolínea.usuarios}[\text{res}[i]]) \geq$
 millasDisponibles (aerolínea.usuarios [res[i+1]])

proc resumenTransacciones (aerolínea: AirMiles, usuario: Usuario) : < IR, IR, IR >
 requiere { usuario $\in$ aerolínea.usuarios }
 asegura { res = < millasTotales (aerolínea.usuarios[usuario]), millasCanjeadas (aerolínea.usuarios[usuario]),
 millasTransferidas (aerolínea.usuarios[usuario]) >

```

proc iniciarPromocion (inout aerolinea : AirMiles, promo : string, factor : R)
  requiere { aerolinea = aerolinea0  $\wedge$  promo  $\notin$  aerolinea0.promos  $\wedge$  factor  $\geq 1$  }
  asegura { aerolinea.usuarios = aerolinea0.usuarios
            aerolinea.promos = setkey(aerolinea0.promos, promo, factor) }

```

aerolinea.promosContope = aerolinea0.promosContope

```

proc iniciarPromocionContope (inout aerolinea : AirMiles, promo : string, factor : R, tope : R)
  requiere { aerolinea = aerolinea0  $\wedge$  promo  $\notin$  aerolinea0.promosContope  $\wedge$  factor  $\geq 1$   $\wedge$  tope  $> 0$  }
  asegura { aerolinea.usuarios = aerolinea0.usuarios
            aerolinea.promosContope = setkey(aerolinea0.promosContope, promo, factor)
            aerolinea.promos = aerolinea0.promos }

```

```

proc finalizarPromocion (inout aerolinea : AirMiles, promo : string)
  requiere { aerolinea = aerolinea0  $\wedge$  promo  $\neq$  'NoPromo' }
  asegura { aerolinea.promos = delkey(aerolinea0.promos, promo)  $\iff$  promo  $\in$  aerolinea0.promos  $\wedge$ 
            aerolinea.promosContope = delkey(aerolinea0.promosContope, promo)  $\iff$  promo  $\in$  aerolinea0.promosContope
             $\wedge$  aerolinea.usuarios = aerolinea0.usuarios }

```

```

proc elQueTieneMasMillas (in aerolinea : AirMiles, in usuario : Usuario) : Usuario
  requiere { usuario  $\in$  aerolinea.usuarios }
  asegura { ( $\forall$  usuario : Usuario) (usuario  $\in$  aerolinea.usuarios  $\wedge$  usuario  $\neq$  res)  $\implies$  (millasTotales(aerolinea.usuarios[res])  $\geq$ 
                                                    millasTotales(aerolinea.usuarios[usuario]) ) }

```

proc elMasCanjeador (in aerolínea : AirMiles, in usuario : Usuario) : Usuario

requiere { usuario $\in$ aerolínea.usuarios }

asegura { ($\forall$ usuario : Usuario) (usuario $\in$ aerolínea.usuarios $\wedge$ usuario $\neq$ res $\rightarrow$

millasCanjeadasAcumuladas(aerolínea.usuarios[res]) $\geq$ millasCanjeadasAcumuladas(aerolínea.usuarios[usuario])

aux millasCanjeadasAcumuladas (transacciones : Transacciones) : Millas =
 SumarTransacciones (transacciones.millasCanjeadas) + SumarTransacciones (transacciones.millasTransferidas.millas) ^{= millas Enviadas}
 millasRecibidas ES seq < struct < transferidos : Usuario, millas : Millas > >
 millasTransferidas ES seq < struct < receptor : Usuario, millas : Millas > >

proc paresTransferidoresExclusivos (in aerolínea : AirMiles) : seq << Usuario, Usuario >>

requiere { True } sinRepetidos (res) $\wedge$

asegura { ($\forall i : \mathbb{Z}$) ($0 \leq i < |res| \rightarrow (res[i]_0 \neq res[i]_1 \wedge sonExclusivos(res[i]_0, res[i]_1, aerolínea.usuarios)) \wedge$

($\forall$ usuarioA, usuarioB : Usuario) (usuarioA $\in$ aerolínea.usuarios $\wedge$ usuarioB $\in$ aerolínea.usuarios $\wedge$ usuarioA $\neq$ usuarioB $\wedge$
 sonExclusivos(A, B, aerolínea.usuarios) $\rightarrow (\exists j : \mathbb{Z}) (0 \leq j < |res| \rightarrow res[j] = \langle usuarioA, usuarioB \rangle)$ }

pred transferidorExclusivo (A : Usuario, B : Usuario, usuarios : Usuarios) :

($\forall i : \mathbb{Z}$) ($0 \leq i < |transferidas| \rightarrow s[i].receptor = B$

pred sinRepetidos (s : seq < tuplas >)
 ($\forall i, j : \mathbb{Z}$) ($0 \leq i, j < |s| \wedge (i \neq j) \rightarrow (s[i] \neq s[j]) \wedge$

$s[i] \neq \langle s[j]_1, s[j]_0 \rangle$

pred sonExclusivos (A : Usuario, B : Usuario, usuarios : Usuarios) :

transferidorExclusivo (A, B, usuarios) $\wedge$ transferidorExclusivo (B, A, usuarios)

$$\text{aux SumarTransacciones (transacción: seq < IR >) : IR} \equiv$$

$$\sum_{i=0}^{\text{transacciones}-1} \text{transacción}[i]$$

$$\text{aux millasTotales (transacciones: Transacciones) : IR} \equiv$$

$$\text{SumarTransacciones (transacciones.MillasGanadas.Millas)} + \text{SumarTransacciones (transacciones.MillasRecibidas)}$$

$$\text{aux millasDisponibles (transacciones: Transacciones) : IR} \equiv$$

$$\text{millasTotales (transacciones)} - \text{SumarTransacciones (transacciones.MillasCanjeadas)} + \text{SumarTransacciones (transacciones.MillasRecibidas)}$$

$$\text{aux millasCanjeadas (transacciones: Transacciones) : IR}$$

$$\text{SumarTransacciones (transacciones.millasCanjeadas)}$$

$$\text{aux millasTransferidas (transacciones: Transacciones) : IR}$$

$$\text{SumarTransacciones (transacciones.millasRecibidas.millas)} + \text{SumarTransacciones (transacciones.millasTransferidas.millas)}$$